AQ-01 · attestqa

Product Document

Version 1.5 (for sign-off) · 2026-10-10 · Author: Vijay Javvadi

1. What we are building

attestqa is an open-source desktop application for Windows. A tester opens it, points it at a website, connects it to an AI model their company already pays for, and it tests that website end to end. The tests it writes are ordinary Playwright code in a folder the tester owns. The part that makes it different from every other "AI testing" tool is that it will not make a failing test pass by weakening the test. That rule is enforced in code, not asked for in a prompt, and every step the AI takes is written to a ledger the tester can audit afterwards.

Releases come at least once a month, starting with v1.0.0 in November 2026 (AQ-05). Each major version marks a new stage: v1 web testing, v2 API testing, v3 the web version of attestqa itself, v4 security testing and v5 ETL testing, all on the same engine and under the same rules. Monthly releases in between (v1.1.0, v1.2.0 …) add features within the stage, and fix releases (v1.2.1 …) go out whenever they are needed. The web version runs the same engine on a company server, so a team can share runs and approvals, and so IDE agents such as GitHub Copilot or Claude Code can call it over MCP.

This document says what the product is and why. How it is built is in the HLD (AQ-03) and LLD (AQ-04). What it looks like is in the Design document (AQ-02).

2. Why it needs to exist

I spent the last stretch of work on agentic-qa, the research runtime behind Paper I. Across 56 recorded runs (6,364 ledger events) the model tried to weaken a test 52 times in 11 runs, by five different routes: rewording an assertion, swallowing the failure in a catch, deleting the check, re-explaining the expected behaviour, and losing an assertion as collateral damage during an unrelated edit. The runtime refused 52 of those attempts. Three got through, each through a gap in what the rules covered, each found by reading the ledger and then closed. The full account is in docs/WEAKENING.md of the agentic-qa repository, frozen at commit `f2771d4` (tag `corpus-2026-10-07`).

That finding is the product. Every AI testing tool on the market will happily turn a red test green. A tester who cannot trust a green result has no result. attestqa sells one thing: when it says a test passed, the test that passed is the test you approved.

3. Who it is for

User	What they need	What attestqa gives them
Manual tester moving into automation	Start from a website, not from a framework	Explore mode turns a site into feature files they can read

User	What they need	What attestqa gives them
Automation engineer	Fast suite growth without losing control of assertions	Specify mode writes Playwright from stories; every change goes through approval
QA lead	Evidence a release is tested, and honest defect records	Attest mode: a counted report, triage verdicts, bugs filed only when a human confirms
Company security / architecture	No new data processor, no new cloud account	Runs on the laptop, uses the company's own model account and sign-in, keeps everything on disk

4. The three modes

Each session is in exactly one mode. The mode decides what the AI may read, what counts as "correct", and what the report is allowed to say.

Mode	Starts from	What "correct" means (the oracle)	Produces	Report may say
Explore	A URL and a test login	Nothing. The agent describes; it does not judge.	Draft `.feature` files, a site map, questions for a human	"This is what the site does." Never "passed" or "bug".
Specify	A story: ADO, Jira, pasted text, or a `.feature` file	The story's acceptance criteria	`features/`, `tests/*.spec.ts`, `pages/*.page.ts` (generated through bdd2pw)	"AC-3 is covered by this test."
Attest	An existing suite	The assertions already written in the suite	A counted run report, triage verdicts, bug records the human confirmed	"12 passed, 2 failed, 2 quarantined — here is the ledger."

The oracle principle is the one rule everything else hangs off: **the agent never supplies its own oracle.** In Explore it has none, and says so. In Specify the oracle is the story. In Attest it is the assertion already in the file. An agent that cannot decide what "correct" means cannot decide that a failing test is wrong, which is exactly the decision the weakening routes depend on.

A new site usually goes Explore → Specify → Attest. Each mode also stands alone: a team with a mature suite starts in Attest. A team with a Selenium suite can start there too: attestqa converts it to Playwright with sel2pw and attests the result.

5. Four test types, one engine

The mode says what the agent may do. The test type says what it is testing. They are independent: any type can be explored, specified or attested. Every type runs through the same governed loop, the same gate, the same ledger and the same oracle principle; what changes is the adapter that does the work, where the oracle comes from, and which weakening moves the refusal rules watch for.

Release	Type	Runs with	Oracle	Weakening moves refused in code
v1.0–v1.4	Web	Playwright, in a browser	Story acceptance criteria	Loosening an assertion, swallowing a failure, deleting a check, and the other routes in the corpus
v2.0–v2.1	API	Playwright's request API (one runner, one report)	Acceptance criteria plus the service's OpenAPI contract	Widening an expected status ("200" to "any 2xx"), dropping schema validation, removing expected fields
v4.0–v4.1	Security	Passive checks in Playwright; active scanning through OWASP ZAP	Acceptance criteria plus the OWASP ASVS requirements the company chooses	Suppressing a finding, adding to an allow list, lowering a severity
v5.0	ETL	Playwright's test runner with a database client; no browser	The source-to-target mapping document	Widening a tolerance, adding a filter that hides bad rows, setting an expected count to whatever came back

Explore means something different for each type, but never includes a verdict. For web it walks pages. For API it reads the OpenAPI file, or the network traffic recorded while exploring the web interface, and drafts request scenarios. For security it lists the attack surface: routes, forms, cookies, headers. For ETL it profiles source and target schemas and drafts reconciliation checks.

Each type also has a hard capability limit that no prompt can change. API calls only reach hosts in scope. Active security scanning is a destructive action: it needs a written authorisation recorded against the target, and is otherwise not offered to the agent at all. The ETL database adapter accepts read queries only and expects read-only credentials.

6. Requirements come from Jira and Azure DevOps

Specify mode builds tests from the stories a team already writes. attestqa connects to an Azure DevOps project or a Jira project with the user's own sign-in, and the user picks the work to cover: a sprint, an epic, a saved query or a JQL filter. Each story's acceptance criteria, comments and linked bugs become the oracle, and a product owner's ruling posted as a comment becomes a citable source (as it did on AB#44 in the Paper I runs).

The Requirements screen shows coverage as a grid: stories down the side, test types across the top, each acceptance criterion marked covered, uncovered or failing, linked to its test and last result. That is the traceability a QA lead needs for a release decision, and it is computed from the ledger and the generated code, not from the model.

Access to the tracker is read-only, with one exception: creating a bug after a person confirms the defect. The agent has no capability to edit a work item's fields or state. A team with no tracker can paste a story or point at a `.feature` file; nothing else changes.

7. What lands on the user's disk

Everything attestqa builds is a normal Playwright project:

```
juice-shop/                       (one folder per site, under TestRunApplication/)
  features/*.feature              Gherkin, from a story or from exploring
  tests/web/  tests/api/  tests/security/  tests/etl/
                                  plain @playwright/test, one folder per test type
  pages/*.page.ts                 page objects (web)
  contracts/                      OpenAPI files and mapping documents used as oracles
  playwright.config.ts
  package.json
  ai-quality.config.yaml          policy: scope, model provider, what the agent may
touch
  .aqa/runs/<run-id>/events.jsonl   the ledger, one per run
```

`npx playwright test` runs that suite with attestqa uninstalled. A team can commit it, run it in their existing CI, and edit it by hand. attestqa governs edits the agent makes while attestqa is running; a person editing in VS Code is never blocked.

What attestqa adds sits beside the code: the ledger, the policy file, and a short provenance comment at the top of each generated spec (run, story, approver). Remove those and the suite is indistinguishable from one written by hand. This is a deliberate difference from hosted tools that keep tests in their own format and rent you a runner.

Two decisions are already agreed and carried into every other document:

- **The tracker is optional.** ADO and Jira are inputs for Specify and destinations for confirmed bugs, nothing more. Without a tracker, a story is pasted text and a bug is a Markdown file under `.aqa/bugs/`. The agent has no capability to edit work-item fields; only bug creation, after human confirmation.
- **The ledger stays on disk.** No attestqa service ever receives a ledger. Teams that want to aggregate can commit `.aqa/` to git.

8. Bring your own model

The company chooses the model. attestqa never resells model access and never needs an attestqa account.

Provider	Sign-in options	Notes
Anthropic	API key	Key stored in the Windows credential vault
OpenAI	API key	Same
Azure OpenAI	API key, or Microsoft Entra ID sign-in	Entra is the usual route for companies standardised on Microsoft and Copilot
AWS Bedrock	AWS SSO / IAM Identity Center profile, or access keys	Claude, Llama and Mistral models behind one API
OpenAI-compatible gateway	Whatever the gateway requires	How a company puts its own proxy, logging and spend limits in front of any model

Provider	Sign-in options	Notes
Local (Ollama, vLLM, LM Studio)	None	Offline use; quality depends on the model

There is no "use my Copilot licence" option. A desktop application cannot borrow a Copilot seat; GitHub licenses its model endpoints to its own clients. A company on Copilot almost always has an Azure tenant, and that is the Azure OpenAI row with Entra sign-in, using the identity already on the laptop.

Every provider adapter must pass the same conformance tests before it ships, so swapping a provider changes the model and nothing else. The ledger records which provider and model produced each decision.

9. Rules the product never breaks

These come from the runtime and the 56-run corpus. The product's job is not to lose them.

1. **A test may be strengthened; it may not be softened.** A change that removes an assertion, or any line that is neither a locator nor a catch, while the test is failing, is refused with the reason. Red stays red until a person rules.
2. **The report is counted, never narrated.** Pass, fail, quarantine, refusal and denial counts come from the ledger. The model's closing note is shown, labelled as the model's note, and never used as a result.
3. **What the agent must not do, it cannot do.** There is no tool to edit a work-item field, no switch to skip governance, no read access to credential files. Capability removal beats instruction.
4. **Every refusal says how to clear it.** A refusal names what would satisfy it (read the whole file; run the whole suite; cite an observation). A refusal only a human can clear is a stall, and a stall is a runtime bug.

10. The interface is review, not chat

There is no free-text box that talks to the model. A chat window invites the user to argue a test into passing, which is the behaviour the product exists to stop. The user reviews what the agent proposes, approves or denies with a reason, and rules on open questions. The one place to type is a question box answered from the ledger ("why is this test quarantined?"), not by the model. The screens are in AQ-02.

11. Open source, desktop first, web second

Licence. MIT, matching the existing runtime. The gate rules, provider adapters and ledger format have to be readable by anyone asked to trust the report, so they are the parts that must be open.

Why desktop first. Credentials never leave the laptop. The browser under test can reach intranet and VPN-only environments. The ledger is disk-only by construction. There is

nothing to host and no account to create. Each of those is a reason an enterprise security review says yes.

The web version is major release v3.0.0, with MCP and multi-user approvals in v3.1.0, straight after API testing (v2), so the server launches with two test types behind it. It reuses the same engine behind a server with company sign-in, the same review screens in a browser, and an MCP endpoint so IDE agents can start runs and read results. It does not change the four rules; it changes who holds the credentials and where the browser runs, which is the decision a company will want to make for itself.

12. Packages we will publish

Package	What it is	Where it is published
`attestqa`	The governed runtime and the `aqa` CLI (exists today as 1.1.0, renamed from `@vijaypjavvadi/agentic-qa`)	npm
`@attestqa/ui`	The review screens as React components, shared by desktop and web	npm
`@attestqa/server`	The web backend and MCP endpoint	npm, plus a container image
`@attestqa/provider-bedrock`, `@attestqa/auth-azure`, `@attestqa/auth-aws`	Optional provider and sign-in plug-ins, kept out of core so core stays small	npm
`@attestqa/tracker-jira`	Jira adapter (Azure DevOps stays in core, which already reads it through Microsoft's Azure DevOps MCP server)	npm
`@attestqa/adapter-zap`, `@attestqa/adapter-sql`	Security scanning through OWASP ZAP; read-only database access for ETL	npm
attestqa for Windows	The signed desktop installer	GitHub Releases

The `@attestqa` npm organisation has to be created before the first scoped publish (decision D4 in AQ-00).

13. Built on our own published packages

attestqa is not written from scratch. Six packages already published under `@vijaypjavvadi` on npm each do one part of the job, and attestqa uses them as ordinary dependencies rather than copying their code. Each keeps its own repository, versions, papers and users, and gains a larger audience through attestqa.

Package	What it does	Role in attestqa	From release
`bdd2pw`	Gherkin `.feature` files to Playwright page objects and specs	Specify mode for web: approved scenarios become code (already a dependency of the runtime)	v1.0
`pw-emit`	Renders Playwright page objects, specs and project scaffolds from a generic intermediate representation	The single code generator for every test type; API, security and ETL tests are emitted through it, so all generated code has one shape	v1.0, extended from v2.0
`synthdata`	Synthetic, relationally consistent test data	Test accounts, form data and API payloads; source data with planted faults for ETL	v1.0, central in v5.0
`pw-self-heal`	Ranks candidate locators with a trained model	Suggests locator repairs to the heal role. Suggest only: every repair is a code change that goes through the gate and a person's approval, and is written to the ledger. Silent healing at run time is switched off in governed runs, because a test that repairs itself without a trace is a test that can pass when it should not	v1.1
`sel2pw`	Migrates Selenium suites (Java, C#) to Playwright	A way in for teams with Selenium suites: convert, then attest	v1.1 (CLI), v1.3 (app)
`usagecontract`	Derives each consumer's contract from observed traffic and checks changes against it	A second oracle for API tests, next to the OpenAPI file, built from the traffic recorded during exploration	v2.1

Three more packages from the same family are relevant. `pw-triage` uses the same failure classes as attestqa's triage and the same event envelope; the two are kept in step rather than allowed to drift. `pw-select` (choose which tests a change affects) fits rerun selection after the first releases. `pw-perf` stays out, with load and performance testing.

14. What success looks like

Every monthly release has its own exit test in AQ-05. Four of them mark a test type as complete, and each repeats the same check for its type.

v1.4.0, web testing complete. A tester on a company laptop installs the signed app, signs in to Azure OpenAI with their work account, explores OWASP Juice Shop to a set of feature files, turns one story from Azure DevOps or Jira into approved Playwright tests, runs them, and gets a report whose numbers match the ledger exactly, with at least one attempted weakening shown as refused. The suite then runs with `npx playwright test` on a machine without attestqa.

v2.1.0, API. The same story gains API tests from Juice Shop's REST interface, checked against its contract. **v3.1.0, web version.** The same run is started from an IDE over MCP and approved by a second person in a browser. **v4.1.0, security.** The protected-route defect from the Paper I runs is found and reported as an access-control failure. **v5.0.0, ETL.** A reconciliation suite between two local synthetic databases catches a seeded transformation fault. In each, at least one attempted weakening of that type is refused and logged.

15. Out of scope

Until their releases: API, security and ETL testing and the web version (dates in AQ-05). For the first year: macOS and Linux builds. Always: load and performance testing, active security scanning of any site without written authorisation from its owner, mobile native apps, and any hosted attestqa service run by us.